application exit, the background threads are interrupted ❻, and then the main thread waits for each background thread to complete before exiting ❼. The background threads sit in a loop, checking for disk changes ❷ and updating the index ❸. Every time around the loop they check for interruption by calling `interruption_point()` ❶.

Why do you interrupt all the threads before waiting for any? Why not interrupt each and then wait for it before moving on to the next? The answer is *concurrency*. Threads will likely not finish immediately when they're interrupted, because they have to proceed to the next interruption point and then run any destructor calls and exception-handling code necessary before they exit. By joining with each thread immediately, you therefore cause the interrupting thread to wait, *even though it still has useful work it could do*—interrupt the other threads. Only when you have no more work to do (all the threads have been interrupted) do you wait. This also allows all the threads being interrupted to process their interruptions in parallel and potentially finish sooner.

This interruption mechanism could easily be extended to add further interruptible calls or to disable interruptions across a specific block of code, but this is left as an exercise for the reader.

9.3. Summary

In this chapter, we've looked at various "advanced" thread-management techniques: thread pools and interrupting threads. You've seen how the use of local work queues and work stealing can reduce the synchronization overhead and potentially improve the throughput of the thread pool and how running other tasks from the queue while waiting for a subtask to complete can eliminate the potential for deadlock.

We've also looked at various ways of allowing one thread to interrupt the processing of another, such as the use of specific interruption points and functions that perform what would otherwise be a blocking wait in a way that can be interrupted.